

Correction

E#4 - Module 4: In-Class Exercises #4

Identify the Components

- **States:** S, ...a, ...b, T, F
 - **Alphabet:** { a, b, }
 - **Start State:** S
 - **Accepting State(s):** F
 - **Transitions:** The list is finalized:
2. S --a--> ...a
 3. S --b--> ...b
 4. S --</>--> T
 5. ...a --</>--> T
 6. ...b --</>--> T
 7. ...a --a--> F
 8. ...a --b--> ...b

9. ...b --a--> ...a

10. ...b --b--> F

Step 1: Confirm States and Symbols

- **States:**
 - S (start)
 - ...a
 - ...b
 - T
 - F (final)
 - **Alphabet:** { a, b, </> }
-

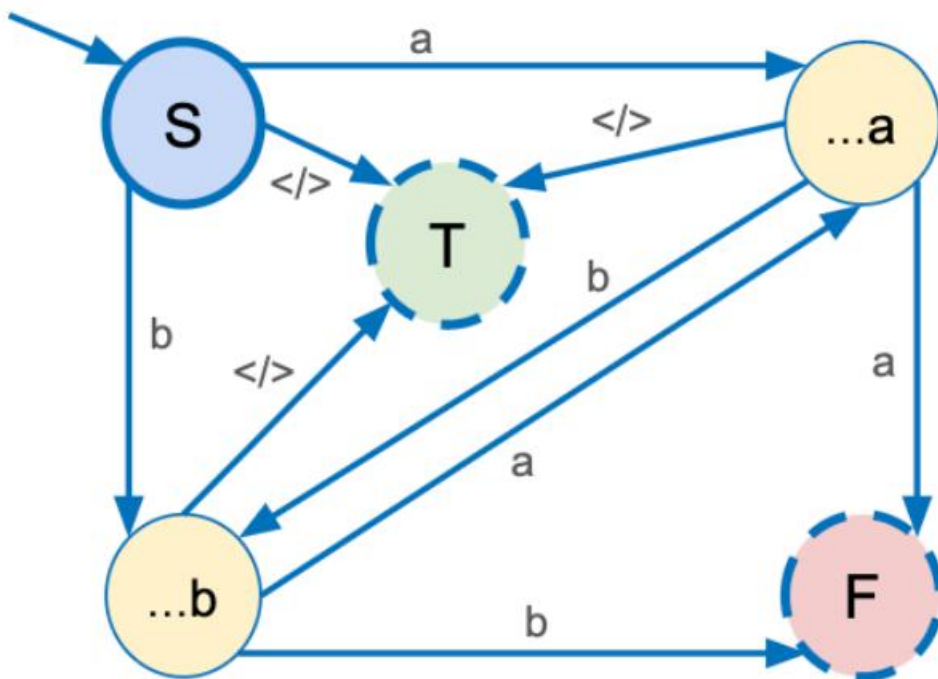
Transition List

1. S --a--> ...a
2. S --b--> ...b
3. S ----> T
4. ...a ----> T
5. ...b ----> T
6. ...a --a--> F
7. ...a --b--> ...b
8. ...b --a--> ...a
9. ...b --b--> F

DFA Components

The DFA description.

DFA Components



- **States:**

- S (Start)
 - ...a
 - ...b
 - T
 - F (Final/Accepting)
-

Transition List

1. $S \xrightarrow{a} \dots a$
(From start state S , reading a moves to state $\dots a$.)
 2. $S \xrightarrow{b} \dots b$
(From start state S , reading b moves to state $\dots b$.)
 3. $S \xrightarrow{\epsilon} T$
(From start state S , reading ϵ moves to state T .)
 4. $\dots a \xrightarrow{\epsilon} T$
(From state $\dots a$, reading ϵ moves to state T .)
 5. $\dots b \xrightarrow{\epsilon} T$
(From state $\dots b$, reading ϵ moves to state T .)
 6. $\dots a \xrightarrow{a} F$
(From state $\dots a$, reading another a moves to accepting state F .)
 7. $\dots a \xrightarrow{b} \dots b$
(From state $\dots a$, reading b moves to state $\dots b$.)
 8. $\dots b \xrightarrow{a} \dots a$
(From state $\dots b$, reading a moves to state $\dots a$.)
 9. $\dots b \xrightarrow{b} F$
(From state $\dots b$, reading another b moves to accepting state F .)
-

Transition Behavior

1. Initial Input Handling from Start State S

- $S \xrightarrow{a} \dots a \rightarrow$ begins tracking an a input.
- $S \xrightarrow{b} \dots b \rightarrow$ begins tracking a b input.
- $S \xrightarrow{\epsilon} T \rightarrow$ any non- a/b input immediately leads to trap.

2. Intermediate States $\dots a$ and $\dots b$

- From $\dots a$:
 - $\dots a \xrightarrow{a} F \rightarrow$ two consecutive a s lead to acceptance.

- ...a --b--> ...b → switches to tracking b.
- ...a --</>--> T → invalid input leads to trap.
- From ...b:
 - ...b --b--> F → two consecutive bs lead to acceptance.
 - ...b --a--> ...a → switches to tracking a.
 - ...b --</>--> T → invalid input leads to trap.

3. Trap State T

Once entered, no transitions are defined—it's a dead-end.

State Colors

S :

- **Color:** Light blue
- **Meaning:** This is the initial state (indicated by the incoming arrow from the left).
- **Border Style:** Solid single border

T :

- **Color:** Light green
- It has a **dashed border**.

...a :

- **Color:** Light yellow.
- **Border Style:** Solid single border

...b :

- **Color:** Light yellow.
- **Border Style:** Solid single border

F (Final/Trap State):

- **Color:** Light red/pink
- **Meaning:** Possibly a dead state or a special non-accepting state. It has a **dashed border**.

Alphabet: { a, b, </> }

States: { S, ...a, ...b, T, F }

Start State: S

Accepting State(s): F

Transition Table

State	a	b	</>
-----	-----	-----	-----
S	...a	...b	T
...a	F	...b	T
...b	...a	F	T
T	—	—	—
F	—	—	—

Assign Variables

Let:

- **S** = start state
- **A** = state "...a"
- **B** = state "...b"
- **F** = final state
- **T** = trap

State Language Equations:

Language Equations

Each state's language is formed by concatenating the language of the previous state with the transition symbol:

- **S** = ϵ
(Start state accepts empty string initially)
- **A** = $S \cdot a + B \cdot a$
(I reach A by reading a from S or a from B)
- **B** = $S \cdot b + A \cdot b$
(We reach B by reading b from S or b from A)
- **F** = $A \cdot a + B \cdot b$
(I reach F by reading a from A or b from B)

So the system is:

$$S = \epsilon$$

$$A = S a + B a$$

$$B = S b + A b$$

$$F = A a + B b$$

I start like:

$$S = \epsilon$$

$$A = S a + B a$$

$$B = S b + A b$$

$$F = A a + B b$$

Replace S in A and B

Since $S = \epsilon$, apply identity $\epsilon P = P$:

$$A = \epsilon a + B a \rightarrow A = a + B a$$

Identity applied: $\epsilon P = P$

$$B = \epsilon b + A b \rightarrow B = b + A b$$

Express A and B recursively

Now i have:

$$A = a + B a$$

$$B = b + A b$$

Substitute B into A

Replace **B** in **A**:

$$A = a + (b + A b) a$$

$$A = a + b a + A b a$$

Factor **A** using identity $R = Q + RP \rightarrow R = QP^*$:

$$A = (a + b a) + A (b a)$$

Identity applied: $R = Q + RP \rightarrow R = QP^*$

$$A = (a + b a)(b a)^*$$

Similarly for B

Start with:

$$B = b + A b$$

Replace A:

$$B = b + ((a + b a)(b a)^*) b$$

Factor **b** at the end:

$$B = b + (a + b a)(b a)^* b$$

Compute F

$$F = A a + B b$$

Replace A and B:

$$F = ((a + b a)(b a)^*) a + (b + (a + b a)(b a)^* b) b$$

Simplify each term:

- First part: $(a + b a)(b a)^* a$
- Second part: $b b + (a + b a)(b a)^* b b$

Combine:

$$F = (a + b a)(b a)^* a + b b + (a + b a)(b a)^* b b$$

Factor common $(a + b a)(b a)^*$:

$$F = (a + b a)(b a)^*(a + b b) + b b$$

Apply union identity:

$$F = (a + b a)(b a)^*(a + b b)$$

Identity applied: Union and concatenation simplification

Final RegEx:

$(a + b a)(b a)^*(a + b b)$

Or in standard notation:

$(a \mid ba)(ba)^*(a \mid bb)$

Derive Final RegEx

After applying all identities and substitutions, I reached:

$F = (a + b a)(b a)^*(a + b b)$

Converting to Standard RegEx Notation

- Replace + with union operator $\mid$.
- Concatenation stays as is.
- Kleene star remains $*$.

So:

$(a \mid ba)(ba)^*(a \mid bb)$

Interpret the Expression

- $(a \mid ba) \rightarrow$ The string starts with either a or ba.
- $(ba)^* \rightarrow$ Zero or more repetitions of ba (alternating pattern).
- $(a \mid bb) \rightarrow$ Ends with either a or bb.

This matches the DFA's behavior:

- Accepts strings with **two consecutive identical letters** (aa or bb), but allows alternating sequences before that.

Deriving Final RegEx:

Final RegEx:

`(a | ba)(ba)*(a | bb)`

It is equivalent to the given DFA and represents the simplest form of the language accepted by the automaton. It ensures that the DFA's behavior—accepting strings with two consecutive identical letters (aa or bb) while allowing alternating sequences—is fully captured.